Lab Sheet 5

Database Management System Lab (BTCS203P)

B. TECH AI&ML, CYBER SECURITY, Data Science 3RD Semester (Session 2026-2025)

Objective:

To understand and practice SQL Joins.

Prerequisites:

1. Basic knowledge of SQL.
2. MySQL installed and set up.

Lab Sheet 5: SQL Joins

```
CREATE DATABASE IF NOT EXISTS LabSheet5DB;
USE LabSheet5DB;

-- Drop tables if they exist to allow clean creation
DROP TABLE IF EXISTS Student;
DROP TABLE IF EXISTS Employee;
DROP TABLE IF EXISTS Course;
DROP TABLE IF EXISTS Department;

-- 1. Department Table
CREATE TABLE Department (
    DepartmentID INT PRIMARY KEY,
    DepartmentName VARCHAR(50)
);

-- 2. Course Table
CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);

-- 3. Student Table
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
```

```
    DepartmentID INT,
    CourseID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

-- 4. Employee Table (with ManagerID referencing EmployeeID)
CREATE TABLE Employee (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DepartmentID INT,
    Salary DECIMAL(10, 2),
    ManagerID INT
);

-- Insert Sample Data
INSERT INTO Department VALUES
(101, 'Computer Science'),
(102, 'Mechanical'),
(103, 'Electrical'),
(104, 'Civil'); -- Department with no students

INSERT INTO Course VALUES
(201, 'Data Structures', 101),
(202, 'Thermodynamics', 102),
(203, 'Circuit Theory', 103),
(204, 'Web Development', 101);

INSERT INTO Student VALUES
(1, 'Alice', 101, 201),
(2, 'Bob', 101, 204),
(3, 'Charlie', 102, 202),
(4, 'Diana', 103, 203),
(5, 'Ethan', NULL, NULL); -- Student with no assigned department or course

INSERT INTO Employee VALUES
(1, 'John Smith', 101, 60000.00, NULL),  -- Top Manager
```

```
(2, 'Sarah Connor', 101, 45000.00, 1),
(3, 'Mike Ross', 101, 75000.00, 1),
(4, 'Rachel Zane', 102, 52000.00, NULL),
(5, 'Harvey Specter', 102, 120000.00, 4);

-- ============================================================
-- Lab Sheet 5: SQL Joins
-- ============================================================

-- Q1. Demonstrate an INNER JOIN between Student and Department
SELECT *
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

StudentID	StudentName	DepartmentID	CourseID	DepartmentID	DepartmentName
1	Alice	101	201	101	Computer Science
2	Bob	101	204	101	Computer Science
3	Charlie	102	202	102	Mechanical
4	Diana	103	203	103	Electrical

```
-- Q2. Display student names along with their department names using INNER JOIN
SELECT s.StudentName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```

Result Grid | Filter Rows: | Export:

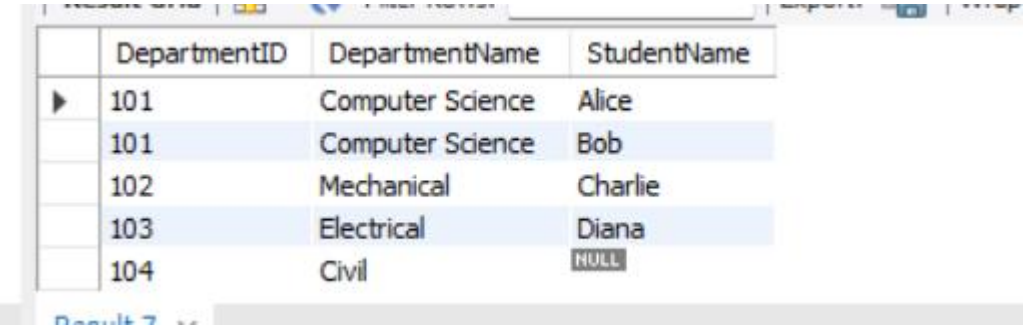
StudentName	CourseName
Alice	Data Structures
Charlie	Thermodynamics
Diana	Circuit Theory
Bob	Web Development

```
-- Q3. Display employee names along with their department names
```

```
SELECT e.EmployeeName, d.DepartmentName
FROM Employee e
INNER JOIN Department d ON e.DepartmentID = d.DepartmentID;

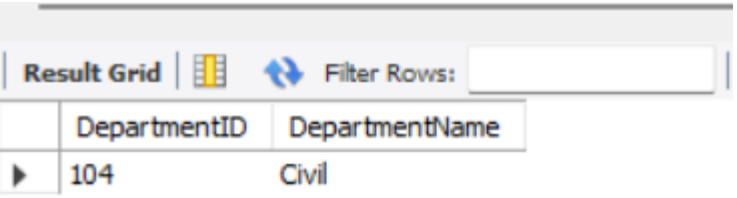
-- Q4. Display students along with their course details
SELECT s.StudentName, c.CourseID, c.CourseName, c.DepartmentID
FROM Student s
INNER JOIN Course c ON s.CourseID = c.CourseID;

-- Q5. Find students belonging to a particular department using INNER JOIN
SELECT s.StudentName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'Computer Science';
```



DepartmentID	DepartmentName	StudentName
101	Computer Science	Alice
101	Computer Science	Bob
102	Mechanical	Charlie
103	Electrical	Diana
104	Civil	NULL

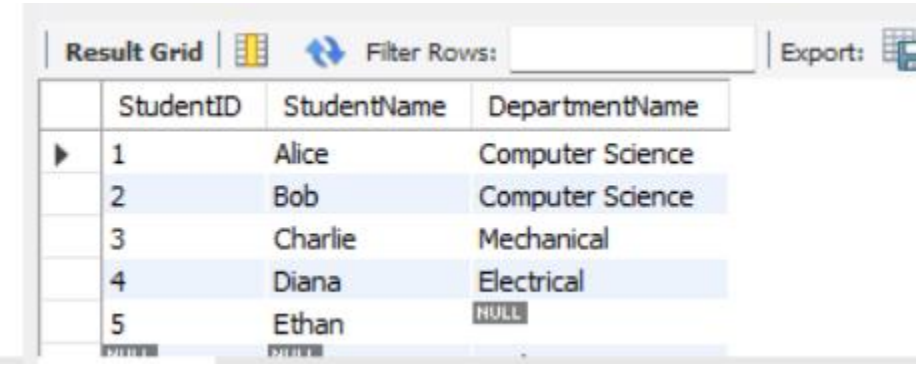
```
-- Q6. Demonstrate a LEFT OUTER JOIN between Student and Department
SELECT *
FROM Student s
LEFT OUTER JOIN Department d ON s.DepartmentID = d.DepartmentID;
```



Result Grid | Filter Rows:

DepartmentID	DepartmentName
104	Civil

```
-- Q7. Display all students, including those who do not have a matching department
SELECT s.StudentID, s.StudentName, d.DepartmentName
FROM Student s
LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID;
```



Result Grid | Filter Rows: | Export:

StudentID	StudentName	DepartmentName
1	Alice	Computer Science
2	Bob	Computer Science
3	Charlie	Mechanical
4	Diana	Electrical
5	Ethan	NULL

```
-- Q8. Demonstrate a RIGHT OUTER JOIN between Student and Department
SELECT *
FROM Student s
RIGHT OUTER JOIN Department d ON s.DepartmentID = d.DepartmentID;

-- Q9. Display all departments, including departments having no students
SELECT d.DepartmentID, d.DepartmentName, s.StudentName
FROM Department d
LEFT JOIN Student s ON d.DepartmentID = s.DepartmentID;

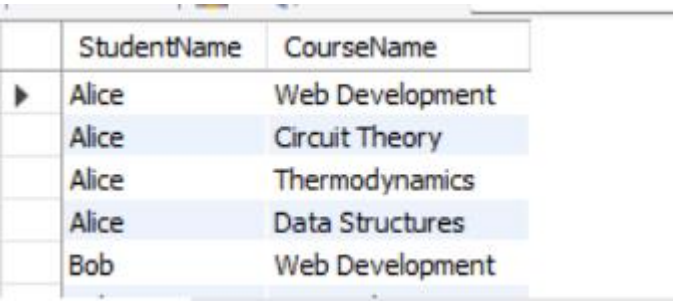
-- Q10. Find students who are not assigned to any department using LEFT JOIN
SELECT s.StudentID, s.StudentName
FROM Student s
LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE d.DepartmentID IS NULL;

-- Q11. Find departments having no students using RIGHT/LEFT JOIN
-- (a) using LEFT JOIN
SELECT d.DepartmentID, d.DepartmentName
FROM Department d
LEFT JOIN Student s ON d.DepartmentID = s.DepartmentID
WHERE s.StudentID IS NULL;

-- (b) using RIGHT JOIN
SELECT d.DepartmentID, d.DepartmentName
FROM Student s
RIGHT JOIN Department d ON s.DepartmentID = d.DepartmentID
WHERE s.StudentID IS NULL;
```

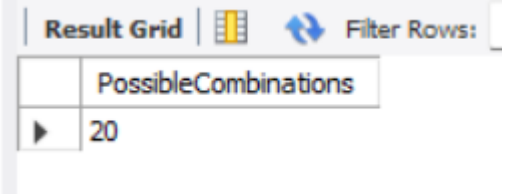
```
-- Q12. FULL OUTER JOIN workaround (MySQL has no FULL JOIN): LEFT JOIN UNION RIGHT JOIN
SELECT s.StudentID, s.StudentName, d.DepartmentID, d.DepartmentName
FROM Student s
LEFT JOIN Department d ON s.DepartmentID = d.DepartmentID
UNION
SELECT s.StudentID, s.StudentName, d.DepartmentID, d.DepartmentName
FROM Student s
RIGHT JOIN Department d ON s.DepartmentID = d.DepartmentID;

-- Q13. Demonstrate a CROSS JOIN between Student and Course
SELECT s.StudentName, c.CourseName
FROM Student s
CROSS JOIN Course c;
```



StudentName	CourseName
Alice	Web Development
Alice	Circuit Theory
Alice	Thermodynamics
Alice	Data Structures
Bob	Web Development

```
-- Q14. Number of possible Student-Course combinations using CROSS JOIN
SELECT COUNT(*) AS PossibleCombinations
FROM Student s
CROSS JOIN Course c;
```



Result Grid | Filter Rows:

PossibleCombinations
20

```
-- Q15. Demonstrate a NATURAL JOIN between two suitable tables
-- (joins automatically on the common column DepartmentID)
SELECT StudentName, DepartmentName
```

```
FROM Student
NATURAL JOIN Department;

-- Q16. Compare the result of NATURAL JOIN and INNER JOIN
-- (a) NATURAL JOIN: common column DepartmentID appears only ONCE (SELECT *)
SELECT *
FROM Student
NATURAL JOIN Department;

-- (b) INNER JOIN ... ON: same rows, but DepartmentID appears TWICE (once per table)
SELECT *
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID;

-- (c) INNER JOIN ... USING: behaves like NATURAL JOIN but you choose the join column
SELECT *
FROM Student
INNER JOIN Department USING (DepartmentID);

-- Q17. SELF JOIN on Employee to show employee-manager relationships
SELECT e.EmployeeName AS Employee, m.EmployeeName AS Manager
FROM Employee e
INNER JOIN Employee m ON e.ManagerID = m.EmployeeID;
```

Result Grid | Filter Rows:

Employee	Manager
Sarah Connor	John Smith
Mike Ross	John Smith
Harvey Specter	Rachel Zane

```
-- Q18. Display employees along with their managers using SELF JOIN
-- (LEFT JOIN so employees without a manager are also shown)
SELECT e.EmployeeName AS Employee, COALESCE(m.EmployeeName, 'No
```

```
Manager') AS Manager
FROM Employee e
LEFT JOIN Employee m ON e.ManagerID = m.EmployeeID;

-- Q19. Join three tables: Student, Department and Course
SELECT *
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
INNER JOIN Course c ON s.CourseID = c.CourseID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

StudentID	StudentName	DepartmentID	CourseID	DepartmentID	DepartmentName	CourseID	CourseName	DepartmentID
1	Alice	101	201	101	Computer Science	201	Data Structures	101
3	Charlie	102	202	102	Mechanical	202	Thermodynamics	102
4	Diana	103	203	103	Electrical	203	Circuit Theory	103
2	Bob	101	204	101	Computer Science	204	Web Development	101

```
-- Q20. Display student name, course name and department name using multiple joins
SELECT s.StudentName, c.CourseName, d.DepartmentName
FROM Student s
INNER JOIN Department d ON s.DepartmentID = d.DepartmentID
INNER JOIN Course c ON s.CourseID = c.CourseID;
```

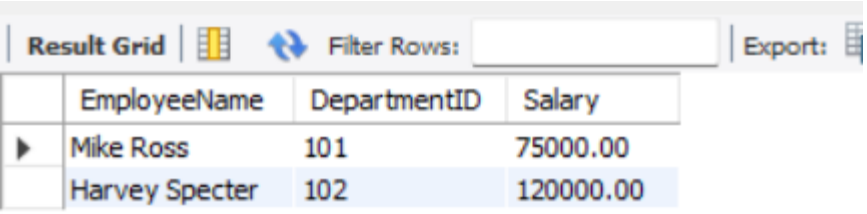
Result Grid | Filter Rows:

DepartmentName	StudentCount
Computer Science	2

```
-- Q21. Employees whose salary is greater than the average salary of their department
-- (a) using a correlated subquery
SELECT e.EmployeeName, e.DepartmentID, e.Salary
FROM Employee e
WHERE e.Salary > (
    SELECT AVG(Salary)
    FROM Employee
```

```
    WHERE DepartmentID = e.DepartmentID
);

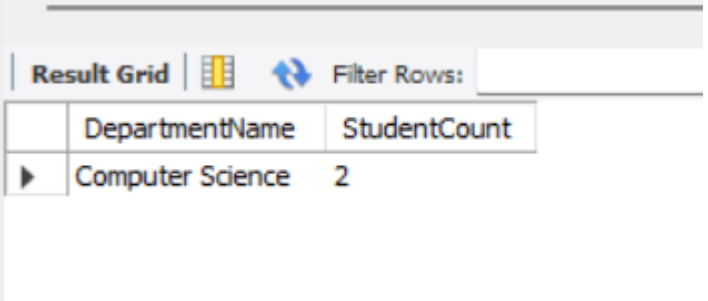
-- (b) using a join with a derived table
SELECT e.EmployeeName, e.DepartmentID, e.Salary, a.AvgSalary
FROM Employee e
INNER JOIN (
    SELECT DepartmentID, AVG(Salary) AS AvgSalary
    FROM Employee
    GROUP BY DepartmentID
) a ON e.DepartmentID = a.DepartmentID
WHERE e.Salary > a.AvgSalary;
```



Result Grid | Filter Rows: | Export:

EmployeeName	DepartmentID	Salary
Mike Ross	101	75000.00
Harvey Specter	102	120000.00

```
-- Q22. Department having the maximum number of students
SELECT d.DepartmentName, COUNT(s.StudentID) AS StudentCount
FROM Department d
INNER JOIN Student s ON d.DepartmentID = s.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName
ORDER BY StudentCount DESC
LIMIT 1;
```



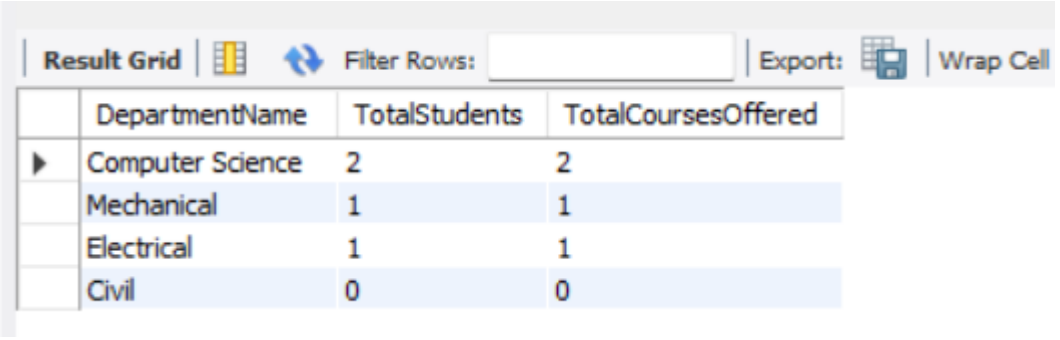
Result Grid | Filter Rows:

DepartmentName	StudentCount
Computer Science	2

```
-- Q23. Departments and the total number of students in each department
SELECT d.DepartmentName, COUNT(s.StudentID) AS TotalStudents
```

```
FROM Department d
LEFT JOIN Student s ON d.DepartmentID = s.DepartmentID
GROUP BY d.DepartmentID, d.DepartmentName;

-- Q24. Courses along with the number of students enrolled in each course
SELECT c.CourseName, COUNT(s.StudentID) AS EnrolledStudents
FROM Course c
LEFT JOIN Student s ON c.CourseID = s.CourseID
GROUP BY c.CourseID, c.CourseName;
```

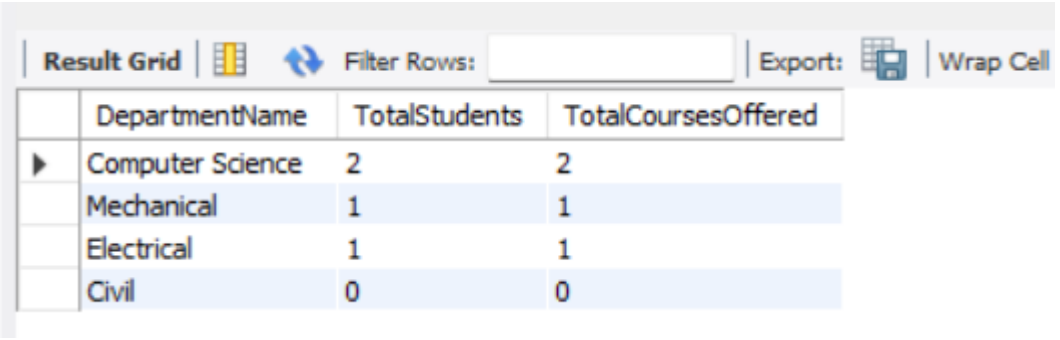


Result Grid | Filter Rows: | Export: | Wrap Cell

DepartmentName	TotalStudents	TotalCoursesOffered
Computer Science	2	2
Mechanical	1	1
Electrical	1	1
Civil	0	0

```
-- Q25. Student-department-course report using INNER JOIN, LEFT JOIN,
--      GROUP BY and aggregate functions
-- INNER JOIN  : Department -> Course (only departments that offer courses)
-- LEFT JOIN   : Student (keeps courses even if nobody is enrolled)
-- GROUP BY    : one row per department/course
-- Aggregates  : COUNT, GROUP_CONCAT
SELECT
    d.DepartmentName,
    c.CourseName,
    COUNT(s.StudentID) AS TotalStudents,
    COALESCE(
        GROUP_CONCAT(s.StudentName ORDER BY s.StudentName SEPARATOR ',
'),
        'No students'
    ) AS StudentNames
FROM Department d
INNER JOIN Course c ON d.DepartmentID = c.DepartmentID
LEFT JOIN Student s ON c.CourseID = s.CourseID
GROUP BY d.DepartmentID, d.DepartmentName, c.CourseID, c.CourseName
```

ORDER BY d.DepartmentName, c.CourseName;


Result Grid | Filter Rows: | Export: | Wrap Cell

DepartmentName	TotalStudents	TotalCoursesOffered
Computer Science	2	2
Mechanical	1	1
Electrical	1	1
Civil	0	0